



PhasorGrid

Relay Compliance Checker

Desktop Application Build Guide

Complete step-by-step instructions to build a production-ready Windows installer (.exe) from scratch — no experience required.

Document Version	1.0
Platform	Windows 10 / 11 (64-bit)
Output	PhasorGrid-Setup-1.0.0.exe (single-file installer)
Estimated Time	45 – 90 minutes (first build)

Table of Contents

- Step 01 Prerequisites & Tools Required
- Step 02 Project Folder Structure
- Step 03 Install Node.js and Electron
- Step 04 Build the Embedded Python Environment
- Step 05 Run the App in Development Mode
- Step 06 Create App Icon
- Step 07 Configure Auto-Updater (GitHub Releases)
- Step 08 Build the Windows Installer (.exe)
- Step 09 Testing the Installer
- Step 10 Auto-Update Workflow
- Step 11 Troubleshooting
- Step 12 Quick Reference Card

Overview

This guide walks you through converting the PhasorGrid web application into a fully self-contained Windows desktop application (.exe installer). The finished product:

- Opens its own window — no browser required
- Bundles Python and Flask inside the installer — nothing extra to install
- Has a branded splash screen, app icon, and professional installer UI
- Supports one-click auto-updates from GitHub Releases
- Works on any Windows 10/11 PC out of the box

■■ The technology stack is: **Electron** (desktop shell) + **Flask** (Python backend) + **Embedded Python** (bundled runtime) + **electron-builder** (packaging) + **electron-updater** (auto-updates). All are free and open-source.

01 Prerequisites & Tools Required

Install each of the following tools on your Windows build machine. You only need to do this once.

Tool	Version	Download URL	Why it's needed
Node.js	20 LTS	https://nodejs.org	Runs Electron and npm
Git	Latest	https://git-scm.com	Version control + GitHub releases
Python	3.11.x	https://python.org/downloads	Local dev testing
VS Code (optional)	Latest	https://code.visualstudio.com	Code editor
GitHub account	—	https://github.com	Auto-update publishing

■ ■ During Node.js install, check 'Automatically install necessary tools' if prompted. During Python install, check 'Add Python to PATH'.

02 Project Folder Structure

Create this exact folder layout on your machine. All files provided in your download correspond to these paths:

```
phasorgrid/ ← Root project folder (create this anywhere)

main.js ← Electron main process

preload.js ← Secure IPC bridge

package.json ← Project config + build settings

requirements.txt ← Python dependencies

build_python_embedded.bat ← Run ONCE to bundle Python

src/

app.py ← Flask backend

index.html ← Frontend UI

assets/

icon.ico ← App icon (Windows, 256x256)

icon.png ← App icon (PNG, 512x512)

tray-icon.png ← System tray icon (32x32)

installer.nsh ← NSIS installer customization

python-embedded/ ← AUTO-CREATED by build_python_embedded.bat

dist/ ← AUTO-CREATED by npm run build (final .exe here)
```

■ Copy all the provided files (.js, .py, .json, .html, .bat, .nsh) into these exact locations before proceeding.

03 Install Node.js and Electron

Open Command Prompt (Win+R → type cmd → Enter) and run these commands from inside the phasorgrid/ folder:

```
REM Navigate to your project folder (adjust path as needed)

cd C:\Users\YourName\Desktop\phasorgrid

REM Install all Electron dependencies listed in package.json

npm install

REM Verify Electron installed correctly

npx electron --version

REM Expected output: v29.x.x
```

■■ npm install will download ~200 MB into a node_modules/ folder. This is normal. Do not commit node_modules/ to Git.

04 Build the Embedded Python Environment

This is the most important step. The .bat script downloads a standalone Python 3.11 embeddable distribution and installs all required packages (Flask, openpyxl, etc.) into it. This Python is what gets bundled inside the installer — users never need to install Python themselves.

```
REM Still in the phasorgrid/ folder:

build_python_embedded.bat

REM The script will:

REM 1. Download python-3.11.8-embed-amd64.zip from python.org

REM 2. Extract it to python-embedded\

REM 3. Install pip into the embedded Python

REM 4. Install flask, flask-cors, olefile, openpyxl

REM When finished you should see:

REM Done! python-embedded\ is ready.
```

■■ Requires internet access during this step. The download is ~25 MB. If behind a corporate proxy, configure your proxy settings first.

■■ The python-embedded/ folder size will be approximately 85-100 MB. This is included inside the final installer automatically.

05 Run the App in Development Mode

Before building the installer, always test the app in development mode:

REM Make sure your local Python is running Flask (for dev testing):

```
pip install flask flask-cors olefile openpyxl
```

REM Start the Electron app (opens the window immediately):

```
npm start
```

REM The app will:

REM 1. Show the PhasorGrid splash screen

REM 2. Start Flask backend on port 5051

REM 3. Open the main window once Flask is ready

■■ In dev mode, main.js looks for src/app.py and src/index.html relative paths. In packaged mode it looks inside process.resourcesPath automatically.

Common dev-mode issues:

- **App shows blank window:** Flask didn't start — check terminal for Python errors
- **Port 5051 already in use:** Kill any existing python processes: `taskkill /f /im python.exe`
- **Module not found errors:** Run: `pip install flask flask-cors olefile openpyxl`

06 Create App Icon

You need three icon files in the assets/ folder. Here's how to create them:

File	Size	Format	How to create
assets/icon.ico	256x256 px	.ico	Use https://icoconvert.com — upload a PNG, download .ico
assets/icon.png	512x512 px	.png	Any PNG with transparent background works
assets/tray-icon.png	32x32 px	.png	Small version of your logo, white/light colored

■ If you do not have icons yet, temporarily create placeholder files. electron-builder will use them during build. Replace with real icons before final release.

REM Quick placeholder icons (PowerShell) – replace later with real ones:

```
powershell -Command "$img=New-Object System.Drawing.Bitmap(256,256);$g=$img.CreateGraphics();$g.FillRectangle([System.Drawing.Brushes]::Navy,0,0,256,256);$img.Save('assets\\icon.png')"
```

REM Convert icon.png to icon.ico using icoconvert.com (free, no signup needed)

07 Configure Auto-Updater (GitHub Releases)

The auto-updater checks GitHub Releases for new versions of the installer. When a new .exe is published on GitHub, users get an in-app notification and can install the update with one click — no manual download needed.

7a. Create a GitHub Repository

1. Go to <https://github.com/new>
2. Repository name: phasorgrid
3. Set to Private (or Public)
4. Click 'Create repository'
5. Note your GitHub username

7b. Update package.json with your GitHub username

In package.json, find the publish section and replace YOUR_GITHUB_USERNAME:

```
"publish": {  
  
  "provider": "github",  
  
  "owner": "YOUR_ACTUAL_GITHUB_USERNAME", ← change this  
  
  "repo": "phasorgrid",  
  
  "releaseType": "release"  
  
}
```

7c. Create a GitHub Personal Access Token

1. Go to: <https://github.com/settings/tokens/new>
2. Note: 'PhasorGrid publish'
3. Expiration: No expiration (or 1 year)
4. Scopes: Check 'repo' (full control)
5. Click 'Generate token' — COPY the token now (shown once only)

6. Set it as an environment variable on your machine:

Windows Command Prompt:

```
setx GH_TOKEN "ghp_YourTokenHere"
```

Then RESTART your Command Prompt for the variable to take effect.

■■ Never paste your GH_TOKEN into source code or commit it to Git. The environment variable approach keeps it secure.

08 Build the Windows Installer (.exe)

With all prerequisites in place, building the installer is a single command:

```
REM Make sure you are in the phasorgrid/ folder

cd C:\Users\YourName\Desktop\phasorgrid

REM Build the Windows x64 installer:

npm run build

REM This will:

REM 1. Package main.js, preload.js, assets/

REM 2. Bundle src/app.py (Flask backend)

REM 3. Bundle src/index.html (frontend)

REM 4. Bundle python-embedded/ (Python runtime)

REM 5. Create NSIS installer

REM 6. Output: dist\PhasorGrid-Setup-1.0.0.exe

REM Build time: 3-8 minutes (first build downloads Electron binaries ~90 MB)
```

■■ The final installer size will be approximately 120-160 MB. This includes Python, Flask, all packages, Electron, and your app code.

Publishing to GitHub (enables auto-updates):

```
REM Build AND publish to GitHub Releases in one step:

set GH_TOKEN=ghp_YourTokenHere

npm run build -- --publish always

REM This creates a GitHub Release and uploads the .exe
```

REM Users running older versions will be notified automatically

09 Testing the Installer

Before distributing, thoroughly test the installer on a clean Windows machine (one that does NOT have Python or Node.js installed):

#	Test	Expected Result
1	Double-click PhasorGrid-Setup-1.0.0.exe	Installer opens with PhasorGrid branding
2	Click through installer wizard	App installs to chosen folder
3	Desktop shortcut appears	PhasorGrid icon visible on desktop
4	Double-click desktop shortcut	Splash screen appears (~2s), then main window
5	Upload a Calc Sheet + RDB file	Run Checks works correctly
6	Multi-File mode with 3 calc sheets	All groups processed and displayed
7	Download PDF report	PDF downloads with correct data
8	Close and reopen app	App reopens cleanly
9	Uninstall via Windows Settings	App removed cleanly, no leftover files

10 Auto-Update Workflow

Once deployed, releasing an update is simple:

```
REM === Releasing a new version ===

REM 1. Update version in package.json and src/app.py:

REM package.json: "version": "1.1.0"

REM src/app.py: APP_VERSION = "1.1.0"


REM 2. Commit your changes:

git add .

git commit -m "Release v1.1.0"

git tag v1.1.0

git push origin main --tags


REM 3. Build and publish:

npm run build -- --publish always


REM 4. Users with v1.0.0 installed will see:

REM 'Update available: v1.1.0 – downloading in background'

REM After download: 'v1.1.0 ready – click Restart to apply update'
```

■ The update banner appears automatically in the app's bottom-right corner. Users click 'Restart' — the app closes, installs the update, and reopens. No manual download or reinstallation needed.

11 Troubleshooting

Issue: npm install fails with EACCES

- Run Command Prompt as Administrator
- Or: `npm install --prefix .` (no admin required in most cases)

Issue: build_python_embedded.bat hangs

- Check internet connection
- Try disabling antivirus temporarily during download
- Manually download Python embed zip from python.org/downloads

Issue: 'electron-builder: not found' error

- Run: `npm install --save-dev electron-builder`
- Or: `npx electron-builder --win`

Issue: App shows 'Backend not reachable'

- Flask failed to start — check logs at: `%APPDATA%\PhasorGrid\logs\main.log`
- Likely cause: python-embedded was not built correctly
- Fix: run `build_python_embedded.bat` again

Issue: GH_TOKEN not recognized

- Restart Command Prompt after running `setx GH_TOKEN`
- Or set it inline: `set GH_TOKEN=your_token && npm run build -- --publish always`

Issue: Installer blocked by Windows Defender

- This is normal for unsigned apps — click 'More info' then 'Run anyway'
- For production release, consider code signing with a certificate from Sectigo/DigiCert

Issue: Auto-update does not trigger

- Ensure the version in `package.json` was actually incremented
- GitHub Release must be published (not a draft)
- Check logs for AutoUpdater errors

12 Quick Reference Card

Keep this card handy for your build workflow:

Task	Command
Install dependencies	<code>npm install</code>
Build embedded Python (once)	<code>build_python_embedded.bat</code>
Run in development	<code>npm start</code>
Build installer only	<code>npm run build</code>
Build + publish to GitHub	<code>npm run build -- --publish always</code>
View app logs	<code>%APPDATA%\PhasorGrid\logs\main.log</code>
Kill stuck Flask process	<code>taskkill /f /im python.exe</code>
Clean build artifacts	<code>rmdir /s /q dist node_modules && npm install</code>
Bump version	Edit 'version' in package.json + APP_VERSION in app.py

■ For code signing (removes 'Windows protected your PC' warning), purchase a Code Signing Certificate from Sectigo (~\$200/yr) and add the 'certificateFile' and 'certificatePassword' fields to the 'win' section of package.json.

For support or questions, consult the project logs first (Help → View Logs inside the app). Most issues are visible there.